

هاكاثون



تِرس

Secupulse

أعضاء الفريق



هتاف السلطان



سارة العطاوي



ديم المفرج

المحتويات:

01	المشكلة	05	الملخص
02	الحل	06	الاختبارات و التحقق
03	جميع البيانات المستخدمة	07	التحديات و الخطط المستقبلية
04	مواعاة الفكرة		

المشكلة

بيانات بطاقات الدفع الحالية ثابتة و لا تموت و بمجرد تسريبها من أي موقع تسوق أو بوابة دفع مخترقة تتحول الى ثغرة مستمرة لسرقة أموال العملاء و نزيف مالي للمنظومة المصرفية.

الحل

بضغطة زر تُصدر بطاقة دفع افتراضية مؤقتة ومخصصة للاستخدام الواحد، مدعومة برمز أمان (Dynamic\ CVV) ينتهي تلقائيًا بعد انتهاء الزمن المخصص لها ؛ مما يضمن إبطال مفعول البيانات وصلاحياتها فورًا و يمنع أي محاولة لإعادة استخدامها من قبل المخترقين .

البيانات المستخدمة

تم استخدام البيانات المولدة و المحاكاة



تم استخدام اللغة البرمجية بايثون



تم استخدام بيانات التحقق و اختبار المنطق



مواءمة الفكرة :

بما أن تجربة العميل هي الركيزة الأهم و الأولوية الأولى في القطاع البنكي، لذا أطلقنا هذه البطاقة المؤقتة لتدمج بين أعلى معايير الحماية و سلسلة الدفع اللحظي، مما يضمن رضا العميل و راحته التامة و حماية أمواله تلقائياً بدون اي تعقيد او مجهود منه.



الملخص

يقدم المشروع حلاً مبتكراً لتعزيز أمان المدفوعات الإلكترونية من خلال إنشاء بطاقات دفع مؤقتة تحتوي على رمز أمان (CVV) ديناميكي، مما يقلل من مخاطر تسريب بيانات البطاقات واستغلالها في عمليات الاحتيال، الهدف من المشروع هو توفير وسيلة آمنة لحماية البيانات من الاحتيال و لقد قمنا بإخراج مخرج أولي عن طريق تطوير كود برمجي ليحاكي الفكرة و نتوقع من هذا المشروع هو تقليل احتمالية استغلال بيانات البطاقات و رفع مستوى الأمان و الثقة في عمليات الدفع .



الاختبار/التحقق:

لقد قمنا بتطوير و تحسين كود برمجي و ملائمته للتطبيق العملي و تم التحقق من كفاءته.

```
import time

class DisposableCard:
    def __init__(self, max_limit):
        self.card_number = "4111222233334444"
        self.max_limit = max_limit # Dynamic limit set by user (e.g., 50 SAR)
        self.dynamic_cvv = "782"
        self.is_active = True
        self.created_at = time.time()

    def process_payment(self, amount, input_cvv):
        # 1. Check card status
        if not self.is_active:
            return "❌ Payment Declined: Card is expired or does not exist!"

        # 2. Check time expiration (e.g., valid for 5 minutes / 300 seconds)
        if time.time() - self.created_at > 300:
            self.is_active = False
            return "❌ Payment Declined: Card session has timed out!"

        # 3. Validate Dynamic CVV
        if input_cvv != self.dynamic_cvv:
            return "❌ Payment Declined: Invalid CVV!"
```

```
# 4. Micro-Rule Engine: Check spending limit
if amount > self.max_limit:
    return f"❌ Payment Declined: Requested amount ({amount} SAR) exceeds card limit ({self.max_limit} SAR)!"

# 5. Success & Self-Destruction
self.is_active = False # The card "dies" instantly here
return f"✅ Payment of {amount} SAR successful. [Security Alert: Card details destroyed immediately]"

# --- Simulation Run ---

# 1. Ahmad generates a card with a 50 SAR limit
my_card = DisposableCard(max_limit=50)

# 2. Ahmad buys the book for 45 SAR (First transaction)
print("--- Transaction 1 (User Buying) ---")
print(my_card.process_payment(amount=45, input_cvv="782"))

# 3. Hacker scenario: Website breached, hacker tries to drain the remaining 5 SAR
print("\n--- Transaction 2 (Hacker Attempt) ---")
print(my_card.process_payment(amount=5, input_cvv="782"))
```

1

الوصول

تأكيد الهوية للوصول



2

تحديد القيود

تحديد المبلغ والوقت للبطاقة



3

عرض البيانات

البطاقة جاهزة للاستخدام



4

إنهاء الصلاحية



التحديات والخطط المستقبلية

ما تحتاج إلى مساعدة فيه:

توفير بيئة تجريبية أو APIs من الجهة البنكية لاختبار الحل.

التحديات:

صعوبة ربط النموذج الأولي مع الأنظمة البنكية بسبب القيود الأمنية.

العرض المستقبلي:

التعاون مع البنك لتطوير الفكرة و تطبيقها على أرض الواقع و توسيع استخدامها.

هاكاثون



شكراً